# CodeLens AI: A Deterministic Program Analysis and AI-Assisted Educational Framework for Python Programming

Shanta Rangaswamy
Department of Computer Science and Engineering
R V College of Engineering
Bengaluru, India
shantharangaswamy@rvce.edu.in

Kala Chandrashekar
Department of Computer Science and Engineering
R V College of Engineering
Bengaluru, India
kalacl@rvce.edu.in

Rohith Arsha
Department of Computer Science and Engineering
R V College of Engineering
Bengaluru, India
rrohitarsha.scs25@rvce.edu.in

Dhruva K
Department of Computer Science and Engineering
R V College of Engineering
Bengaluru, India
dhruvak.scs25@rvce.edu.in

***Abstract*—Learning programming involves more than identifying syntax errors; students must understand how programs execute, why errors occur, and how code can be improved. While traditional development tools effectively detect programming errors, they often provide limited support for explaining program behavior. Recent Large Language Models (LLMs) offer natural language explanations but may generate responses that are difficult to verify when used independently.**

**This paper presents CodeLens AI, an educational programming framework that combines deterministic static analysis with AI-assisted explanations. The framework integrates AST-based analysis, rule-based bug detection, complexity estimation, secure runtime visualization, and automated code correction. Rather than allowing the LLM to analyze source code directly, verified analysis results are used to generate grounded and consistent explanations.**

**Experimental evaluation demonstrates that CodeLens AI accurately analyzes Python programs, visualizes execution, detects common programming errors, applies automated code corrections, and generates contextual explanations that support program comprehension and learning.**

***Index Terms*—Programming Education, Static Program Analysis, Abstract Syntax Tree, Runtime Visualization, Automated Code Correction, Program Comprehension, Explainable Artificial Intelligence, Educational Software Engineering**

## I. Introduction

Programming is a fundamental component of computer science education, making effective programming assistance increasingly important. While modern Integrated Development Environments (IDEs), compilers, and static analysis tools can detect syntax and semantic errors, they often provide limited support for helping students understand program behavior, identify the cause of errors, or improve their coding practices. As a result, many novice programmers rely on trial-and-error debugging rather than developing a deeper understanding of programming concepts.

Recent advances in Large Language Models (LLMs) have introduced new possibilities for programming education through natural language explanations, automated debugging, and interactive tutoring. However, these models rely on probabilistic reasoning and may occasionally produce explanations that are inconsistent with the actual behavior of a program. Conversely, deterministic static analysis provides reliable and reproducible diagnostics but typically lacks interactive explanations and runtime visualization.

To address these limitations, this paper presents **CodeLens AI**, an educational programming assistance framework that combines deterministic program analysis with AI-assisted explanations. The framework integrates AST-based static analysis, secure runtime visualization, automated code correction, complexity analysis, and a locally hosted LLM that generates explanations from verified analysis results rather than directly interpreting source code. To evaluate the proposed framework, a reproducible benchmark consisting of 1,000 Python programs spanning ten categories of programming tasks and programming errors was constructed.

The primary contributions of this work are as follows:

- An integrated educational framework combining static analysis, runtime visualization, automated code correction, complexity analysis, and AI-assisted explanations.
- A secure AST-based runtime interpreter for visualizing Python program execution without relying on `exec()` or `eval()`.
- A deterministic rule-based engine for detecting and correcting common programming errors.
- Integration of a locally hosted LLM that generates explanations from verified analysis results.
- A reproducible benchmark of 1,000 Python programs for quantitative evaluation of the proposed framework.

The remainder of this paper is organized as follows. Section II reviews related work, Section III presents the system architecture, Section IV describes the methodology and implementation, Section V discusses the experimental results, and Section VI concludes the paper.

## II. Related Work

Research on programming assistance has evolved from traditional rule-based analysis to modern approaches that leverage semantic representations, transformer-based models, and automated software engineering techniques. Although these methods have significantly improved individual programming tasks, most existing systems concentrate on solving a specific problem rather than providing a comprehensive educational environment.

One of the earliest directions focused on automated assessment of programming assignments. Xiangmin and Lin [1] proposed an intelligent scoring method for C programs by combining static and dynamic semantic analysis to improve evaluation accuracy. Zen *et al.* [2] explored the use of Latent Semantic Analysis (LSA) to grade programming assignments by measuring semantic similarity between student submissions and reference solutions. More recently, Narmada and Pati [3] showed that transformer-based code embeddings generated by UniXcoder capture program semantics more effectively, resulting in improved automated grading performance across different coding perspectives.

Several researchers have investigated semantic understanding of source code beyond grading applications. Arora *et al.* [4] combined static program analysis with neural networks to classify programs according to their functional behavior. Zhang *et al.* [5] introduced an intermediate representation that bridges source code and binary code, enabling semantic understanding across different execution environments. Cherradi and El Mahajer [6] addressed software documentation by fine-tuning CodeT5 to generate descriptive comments for Java methods, producing contextually relevant documentation from source code.

Large Language Models and transformer architectures have also become increasingly important in software engineering. Rafique *et al.* [7] compared multiple transformer models for automated unit test generation and reported that CodeT5 consistently generated higher-quality executable test cases. Tufano *et al.* [8] proposed pretrained transformer models for automatic assertion generation, reducing the manual effort required to develop software tests. Shin *et al.* [9] further improved test generation through domain adaptation techniques that enhanced model generalization. Pynguin, introduced by Lukasczyk and Fraser [10], combined search-based software testing with program analysis for automatic Python unit test generation. AthenaTest [11] demonstrated another transformer-based approach for producing unit tests, while Alagarsamy *et al.* [12] improved assertion generation using verification and domain adaptation strategies. EvoSuite [13] remains one of the most widely adopted search-based frameworks for automatic test suite generation, whereas PyTester [14] incorporated reinforcement learning to translate natural language descriptions into executable test cases.

Another important research direction focuses on improving program comprehension through semantic analysis. Zhao *et al.* [15] proposed a semantic-aligned code summarization framework that combines Abstract Syntax Trees (ASTs) with dataflow information to produce more accurate natural language summaries. Their work illustrates how integrating structural and semantic program information can substantially improve code understanding.

Despite these contributions, existing studies generally address isolated problems such as automated grading, code summarization, documentation, semantic analysis, or software testing. Only limited attention has been given to integrating these capabilities into a single educational programming framework. The proposed CodeLens AI framework addresses this gap by combining deterministic AST-based analysis, secure runtime visualization, automated code correction, complexity analysis, and AI-assisted explanations within a unified system. By generating explanations from verified analysis results instead of directly interpreting source code, the framework aims to provide feedback that is both reliable and educationally meaningful.

## III. System Architecture

The overall architecture of the proposed CodeLens AI framework is illustrated in Fig. 1. The framework adopts a modular pipeline that combines deterministic program analysis with AI-assisted educational feedback. Instead of allowing a Large Language Model (LLM) to independently interpret source code, CodeLens AI first performs a series of deterministic analyses to extract verified program information. The validated results are then used to construct contextual prompts for a locally hosted LLM, ensuring that generated explanations remain consistent with the analysis results.

As shown in Fig. 1, Python source code is initially parsed into an Abstract Syntax Tree (AST), which forms the foundation of the analysis pipeline. The Semantic Extractor traverses the AST to collect structural information including variables, functions, loops, conditional statements, imports, and other language constructs. This intermediate representation is subsequently processed by multiple analysis modules operating independently on the extracted program information.

The Rule Engine performs deterministic static analysis to identify common programming errors such as undefined variables, infinite loops, missing recursion base cases, unreachable code, and incorrect algorithm implementations. In parallel, the Runtime Engine safely interprets supported Python programs using a restricted AST interpreter without relying on Python's native `exec()` or `eval()` functions. The Complexity Engine estimates program metrics including time complexity, space complexity, cyclomatic complexity, maintainability score, and other structural statistics. Additional modules analyze import statements, function dependencies, and potentially unsafe programming constructs.

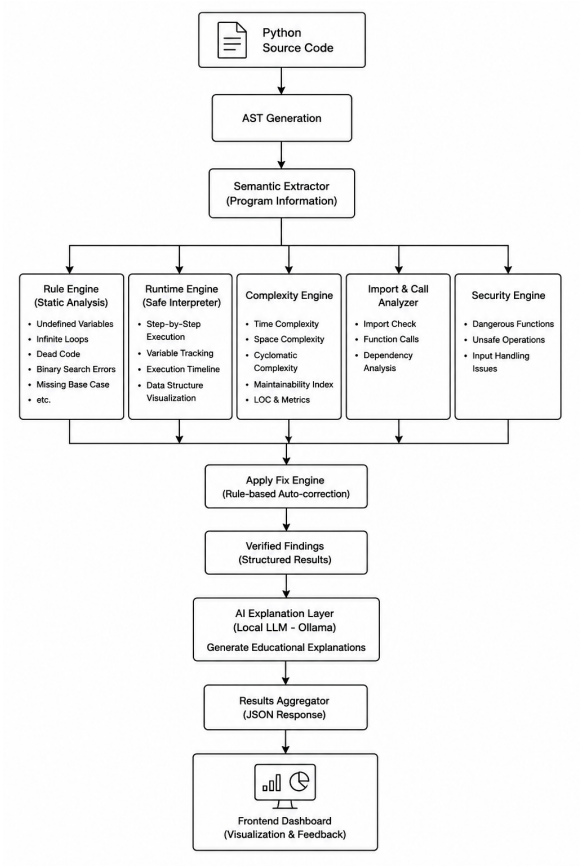


Fig. 1. Overall architecture of the proposed CodeLens AI framework.

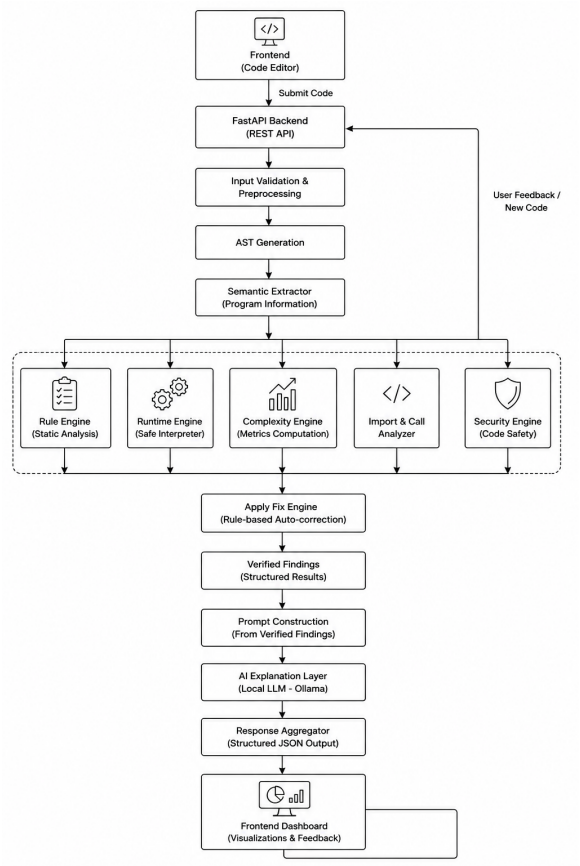


Fig. 2. Workflow of the proposed CodeLens AI analysis pipeline.

The outputs produced by these analysis modules are consolidated by the Apply Fix Engine, which performs deterministic rule-based corrections for supported programming errors. The verified analysis results are then forwarded to the AI Explanation Layer, where a locally hosted Ollama language model generates contextual educational explanations. Since the language model receives verified findings rather than raw source code, the generated feedback remains grounded in deterministic analysis.

The interaction between individual components is illustrated in Fig. 2. After the user submits a Python program through the frontend editor, the request is processed by the FastAPI backend, where input validation and preprocessing are performed before the analysis pipeline begins. Each analysis module contributes independent observations that are combined into a structured response before being presented through the user interface.

The workflow emphasizes modularity and separation of responsibilities. Static analysis, runtime interpretation, complexity estimation, automated code correction, and AI-assisted explanation are implemented as independent components that communicate through structured data rather than direct dependencies. This design simplifies maintenance, allows individual modules to evolve independently, and enables additional analysis components to be incorporated with minimal modification to the existing architecture. The frontend dashboard finally presents the analysis results, runtime visualization, complexity metrics, detected issues, suggested corrections, and AI-generated explanations within a unified interactive environment.

## IV. Methodology

The proposed CodeLens AI framework follows a multi-stage analysis pipeline that processes Python programs and generates educational feedback through a combination of rule-based analysis and AI-assisted explanations. All program analysis is completed before invoking the language model, ensuring that generated explanations remain grounded in verified analysis results. The overall processing workflow is illustrated in Fig. 2.

### *A. Source Code Parsing*

The analysis begins by validating the submitted Python program and parsing it into an Abstract Syntax Tree (AST) using Python's built-in `ast` module. The AST preserves the structural relationships between language constructs while eliminating formatting information, providing a standardized intermediate representation for subsequent analysis.

### B. Semantic Extraction

The generated AST is traversed to extract semantic information required by the analysis pipeline. During this stage, the Semantic Extractor identifies variables, functions, assignments, loops, conditional statements, recursive calls, imports, function invocations, and other structural elements. The extracted information is organized into an intermediate representation that is shared across the remaining analysis modules.

### C. Static Analysis

Next, the Rule Engine evaluates the extracted semantic information using predefined analysis rules. Since the framework follows a deterministic analysis strategy, identical source programs always produce identical analysis results. The current implementation detects common programming issues including undefined variables, infinite loops, unreachable code, missing recursion base cases, incorrect binary search implementations, missing return statements, variable shadowing, dangerous imports, and several additional programming mistakes frequently encountered by novice programmers.

### D. Runtime Interpretation

To improve program comprehension, CodeLens AI includes a secure runtime interpreter capable of executing supported Python programs without relying on Python's native `exec()` or `eval()` functions. Instead, the interpreter evaluates the AST directly while recording variable updates, execution steps, return values, and console output to construct an execution timeline. This restricted execution model improves safety while providing sufficient functionality for educational demonstrations and algorithm visualization.

### E. Complexity Analysis

The Complexity Engine estimates several structural metrics using information extracted from the AST. These include estimated time complexity, space complexity, cyclomatic complexity, maintainability score, function count, loop count, and other source code statistics. The computed metrics provide learners with quantitative insights into program quality and algorithmic efficiency.

### F. Automated Code Correction

Once the analysis is complete, the detected issues are passed to the Apply Fix Engine, which applies predefined rule-based transformations to correct supported programming errors. The current implementation can resolve issues such as undefined variable references, missing recursion base cases, selected binary search implementation errors, and unreachable code. After these corrections are applied, the updated program can be analyzed again to confirm that the identified issues have been resolved successfully.

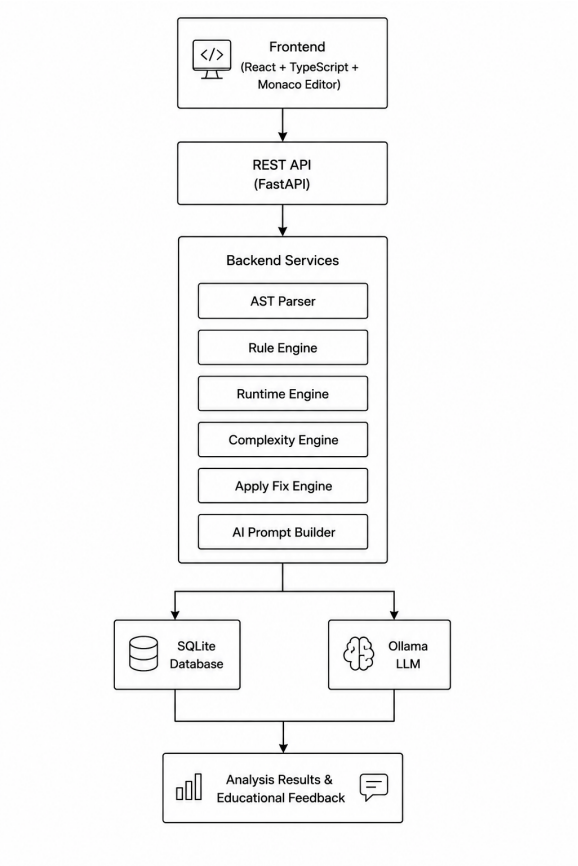


Fig. 3. Implementation stack of the proposed CodeLens AI framework.

### G. AI-Assisted Explanation Generation

The final stage of the pipeline generates educational explanations using a locally hosted Ollama language model. Instead of allowing the model to interpret the source code directly, CodeLens AI supplies it with structured outputs from the analysis pipeline, including detected issues, runtime observations, complexity metrics, and any applied code corrections. Using verified analysis results as input helps keep the generated explanations accurate, consistent, and easier for learners to understand.

## V. Implementation Details

CodeLens AI was developed as a modular client–server application that separates the user interface from the program analysis engine. Figure 3 illustrates the implementation stack. The frontend provides an interactive coding environment, whereas the backend is responsible for parsing source code, performing program analysis, generating runtime visualizations, applying automated code corrections, and producing AI-assisted explanations.

### A. Frontend Implementation

The user interface was built using React and TypeScript to provide a responsive web-based programming environment. Monaco Editor was integrated to offer features commonly

found in modern IDEs, including syntax highlighting, automatic indentation, code formatting, and line numbering. User requests and analysis results are exchanged with the backend through RESTful APIs using JSON, allowing seamless communication between both components.

### B. Backend Implementation

FastAPI was selected for the backend because it offers efficient asynchronous request handling while keeping the overall architecture lightweight. The analysis pipeline is organized into independent modules that operate on a shared Abstract Syntax Tree (AST) representation. This modular design allows new analysis capabilities to be added without requiring significant changes to the existing implementation.

### C. Analysis Components

The backend is composed of several independent components that perform source code parsing, static analysis, runtime interpretation, complexity estimation, automated code correction, and explanation generation. Rather than communicating directly with one another, these components exchange structured intermediate data generated during the analysis process. This approach reduces coupling between modules, simplifies maintenance, and makes it easier to extend the framework with additional analysis features.

### D. Data Management and AI Integration

Analysis reports are stored in an SQLite database, enabling previous sessions to be retrieved whenever required. Educational explanations are generated using a locally hosted Ollama language model. Instead of sending the original source code to the model, CodeLens AI provides verified outputs from the analysis pipeline, including detected issues and program metrics. This approach keeps the generated explanations consistent with the analysis results while also preserving user privacy by avoiding cloud-based inference.

### E. Software Quality Assurance

Software reliability was evaluated through automated unit testing of the analysis modules, runtime interpreter, and backend services. The implementation successfully passed 128 automated unit tests. In addition, Ruff was used for static code quality analysis, MyPy for static type checking, and a reproducible benchmark consisting of 1,000 Python programs was used to quantitatively evaluate the detection capabilities of the proposed framework. The modular implementation also enables additional analysis rules and educational features to be incorporated with minimal changes to the existing architecture.

## VI. Results and Discussion

The proposed CodeLens AI framework was evaluated using a synthetic benchmark comprising 1,000 Python programs spanning ten representative categories of programming tasks and common programming errors. The benchmark included both correct implementations and programs containing undefined variables, infinite loops, missing recursion base cases, binary search logic errors, unreachable code, unused variables, missing return statements, variable shadowing, and dangerous imports. Each program was assigned a ground-truth label and analyzed using the deterministic rule-based analysis engine integrated within CodeLens AI.

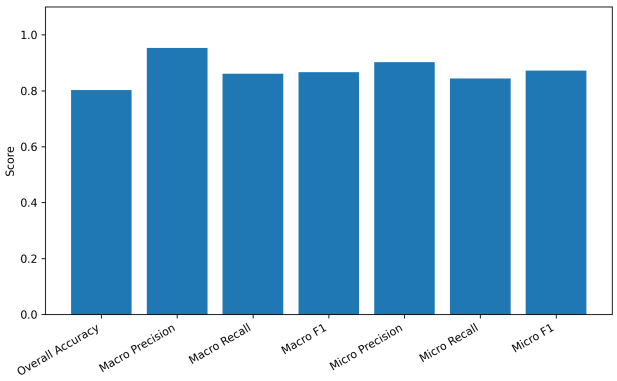


Fig. 4. Overall evaluation metrics obtained using the proposed benchmark dataset.

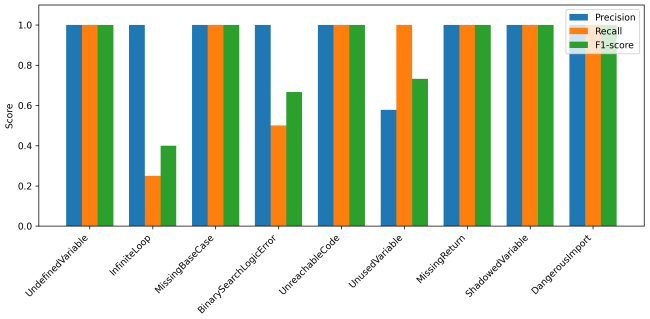


Fig. 5. Rule-wise precision, recall, and F1-score of the proposed CodeLens AI framework.

Figure 4 summarizes the overall evaluation metrics obtained from the benchmark. The framework achieved an overall accuracy of 80.2%, with a macro precision of 95.31%, macro recall of 86.11%, and macro F1-score of 86.66%. The corresponding micro precision, recall, and F1-score were 90.24%, 84.38%, and 87.21%, respectively. These results indicate that the proposed framework produces highly reliable detections with relatively few false positives while maintaining strong overall detection performance across multiple categories of programming errors.

The rule-wise evaluation is presented in Fig. 5. Six rule categories, namely Undefined Variable, Missing Base Case, Unreachable Code, Missing Return, Shadowed Variable, and Dangerous Import, achieved perfect precision, recall, and F1-scores. These results demonstrate the effectiveness of deterministic Abstract Syntax Tree (AST)-based analysis for identifying structurally verifiable programming errors. Since these rules depend on well-defined syntactic and semantic properties, the proposed rule engine consistently produced accurate detections across the benchmark.

Lower recall was observed for Infinite Loop detection and

TABLE I
COMPARISON OF REPRESENTATIVE APPROACHES WITH THE PROPOSED CODELENS AI FRAMEWORK.

| Reference | Primary Contribution |
|---|---|
| [4] | Semantic program detection using static and dynamic analysis for program classification. |
| [6] | Automated Java method comment generation using a fine-tuned CodeT5 model. |
| [15] | Semantic code summarization through ASTs and data-flow analysis to improve code understanding. |
| [7] | Transformer-based automatic unit test generation for Python programs. |
| **CodeLens AI** | An integrated educational programming framework combining deterministic AST-based static analysis, secure runtime visualization, automated code correction, complexity estimation, and grounded AI-assisted explanations using a locally hosted language model. |

Binary Search Logic Error detection. Although both rules achieved perfect precision, their recall values were comparatively lower because many logically incorrect implementations cannot be conclusively identified through local syntactic analysis alone. Detecting such errors often requires deeper reasoning about control flow, variable updates, loop termination conditions, and algorithmic intent. Similarly, the Unused Variable rule achieved perfect recall but comparatively lower precision due to conservative detection behavior, where several variables that contributed indirectly to program execution were classified as unused. These observations highlight the inherent limitations of purely deterministic static analysis when reasoning about complex program semantics.

Table I highlights the primary focus of representative approaches in comparison with the proposed framework. Existing studies have primarily addressed individual software engineering tasks, including semantic program detection, automated documentation, code summarization, and unit test generation. In contrast, CodeLens AI unifies deterministic static analysis, runtime visualization, automated code correction, complexity analysis, and AI-assisted educational explanations within a single architecture. By grounding language-model explanations in verified analysis results rather than directly interpreting source code, the proposed framework provides reliable, explainable, and educationally meaningful programming assistance.

Overall, the experimental results indicate that CodeLens AI provides reliable support for analyzing Python programs while also offering meaningful educational feedback. The benchmark shows that the framework performs well across several categories of programming errors, and the comparison with related work illustrates that it brings together capabilities that are typically addressed separately in existing systems. Because the architecture is modular, new analysis rules and learning features can be added with relatively little effort, making the framework well suited for future development.

## VII. CONCLUSION AND FUTURE WORK

This paper introduced **CodeLens AI**, an educational programming framework that combines deterministic program analysis with AI-assisted explanations to support programming education. The framework brings together AST-based static analysis, secure runtime visualization, automated code correction, complexity estimation, and a locally hosted language model within a single environment. Instead of relying entirely on the reasoning capabilities of an LLM, CodeLens AI generates explanations from verified analysis results, making the feedback more consistent, transparent, and trustworthy for learners.

To evaluate the framework, a benchmark of 1,000 Python programs covering ten categories of common programming tasks and errors was developed. The evaluation produced an overall accuracy of 80.2%, with a macro precision of 95.31% and a macro F1-score of 86.66%. The rule-based analysis engine achieved perfect precision and recall for several deterministic programming errors, including undefined variables, missing recursion base cases, unreachable code, missing return statements, variable shadowing, and dangerous imports. Lower recall for infinite loop detection and binary search logic analysis indicates that these problems often require reasoning beyond syntactic structure and remain challenging for purely static analysis techniques.

The results suggest that deterministic analysis and AI-generated educational feedback complement each other effectively. While the analysis engine provides reliable and reproducible program diagnostics, the language model helps present these findings in a form that is easier for students to understand. The modular design of the framework also makes it straightforward to introduce additional analysis rules or educational features without requiring major changes to the overall architecture.

Future work will focus on strengthening the framework's ability to analyze complex program logic by incorporating control-flow analysis, semantic graph representations, and interprocedural analysis techniques. Additional improvements include extending support to multiple programming languages, expanding the benchmark with real student submissions, and developing adaptive feedback mechanisms that tailor explanations to a learner's experience level. These enhancements aim to make CodeLens AI a more comprehensive and intelligent programming assistant for software engineering education.

## REFERENCES


[1] W. Xiangmin and H. Lin, "The Research of C Language Programming Intelligent Scoring Technology Based on the Semantic Similarity," in *Proc. IEEE*, 2012.

[2] K. Zen, D. N. F. Awang Iskandar, and O. Linang, "Using Latent Semantic Analysis for Automated Grading Programming Assignments," in *2011 International Conference on Semantic Technology and Information Retrieval (STAIR)*, 2011.

[3] N. Narmada and P. B. Pati, "Evaluating uniXcoder Embeddings for Automated Grading: A Study Across Varied Code Perspectives," in *2024 15th International Conference on Computing Communication and Networking Technologies (ICCCNT)*, 2024.

[4] B. Arora, S. V. C., G. R. Dheemanth, M. Thakral, and N. S. Kumar, "Code Semantic Detection," in *2021 Asian Conference on Innovation in Technology (ASIANCON)*, 2021.

[5] Z. Zhang, S. Liu, Q. Yang, and S. Guo, "Semantic Understanding of Source and Binary Code Based on Natural Language Processing," in *2021 IEEE 4th Advanced Information Management, Communicates, Electronic and Automation Control Conference (IMCEC)*, 2021.

[6] M. Cherradi and H. El Mahajer, "Semantic-Driven Automated Comment Generation for Java Methods Using Fine-Tuned CodeT5," in *2025 8th International Conference on Networking, Intelligent Systems & Security (NISS)*, 2025.

[7] K. A. Rafique, P. Biradar, and C. Grimm, "Transformer-Based Unit Test Generation," in *2025 IEEE Conference on Artificial Intelligence (CAI)*, 2025.

[8] M. Tufano *et al.*, "Generating Accurate Assert Statements for Unit Test Cases Using Pretrained Transformers," in *Proc. IEEE/ACM International Conference on Automation of Software Test*, 2022.

[9] J. Shin, S. Hashtroudi, H. Hemmati, and S. Wang, "Domain Adaptation with Transformer Models for Test Case Generation," 2023.

[10] S. Lukasczyk and G. Fraser, "Pynguin: Automated Unit Test Generation for Python," arXiv preprint, 2022.

[11] M. Tufano, D. Drain, A. Svyatkovskiy, S. K. Deng, and N. Sundaresan, "AthenaTest: Transformer-Based Unit Test Case Generation," Microsoft Research, 2021.

[12] S. Alagarsamy, C. Tantithamthavorn, and A. Aleti, "A3Test: Assertion-Driven Test Case Generation with Domain Adaptation," Monash University, 2023.

[13] G. Fraser and A. Arcuri, "EvoSuite: Automatic Test Suite Generation for Object-Oriented Software," in *Proc. European Software Engineering Conference and ACM SIGSOFT Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2011.

[14] W. Takerngsaksiri, R. Charakorn, C. Tantithamthavorn, and Y.-F. Li, "PyTester: A Reinforcement Learning-Based Text-to-Testcase Generation Framework," 2023.

[15] Y. Zhao, Z. Huang, K. Zhang, W. Gao, Q. Liu, X. Liu, F. Yao, and E. Chen, "Semantic-Aligned Code Summarization: Bridging the Gap Between Code and Natural Language Through Data Flow Analysis," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 36, no. 10, pp. 19240–19253, Oct. 2025.